

Both are worth looking at if you're starting fresh. For most existing teams, Airflow's ecosystem and community make it hard to beat.

The micro-batching middle ground

I want to spend a moment on micro-batching because it's where a lot of real-world systems live, and it's underrated.

The idea is simple. Instead of true streaming with sub-second latency, you run small batch jobs every few seconds or minutes. You get most of the freshness benefits of streaming. You get most of the simplicity of the batch. You skip a lot of the hardest parts of true streaming.

Spark Structured Streaming also leans heavily on micro-batching by default. So does much of dbt's streaming work. The pattern is common enough that I'd say it's the default approach unless you specifically need sub-second latency.

Don't dismiss micro-batching as "not real streaming." For most business use cases, it's the right answer. Reserve true streaming for the cases where you really, genuinely cannot wait sixty seconds.

A real-world example, end-to-end

Let me walk through what a hybrid batch and streaming pipeline might look like in practice.

Imagine you run an e-commerce site. You want a few things. You want a real-time dashboard for the marketing team showing live conversion rates. You want operational alerts when something is going wrong. And you want a nightly data warehouse refresh for proper analytics.

For the real-time dashboard, you set up Debezium to stream changes from your orders database into Kafka. A Flink job reads those events, calculates rolling conversion rates by traffic source, and writes the results to a Redis cache that the dashboard reads from. End-to-end latency in a few seconds.

For the operational alerts, the same Kafka events feed a separate Flink job that watches for anomalies. Sudden drops in successful orders. Spikes in error rates. The job pushes alerts to Slack and PagerDuty.

For the nightly warehouse, an Airflow DAG runs at 2 a.m. It reads the previous day's data from your operational database, runs heavy transformations in dbt or Spark, and loads it into Snowflake or BigQuery. This is the source of truth for proper analytics, finance reports, and ML model training.

Notice that all three pipelines coexist. Streaming for things that need to be fresh. Batch for things that need to be thorough. Each one is the right tool for its job. The streaming pipeline doesn't try to do everything. The batch pipeline doesn't get stretched into territory it isn't suited for.

This kind of mixed architecture is what most mature data platforms look like in practice. Pure-streaming and pure-batch are the extremes. The middle is where real systems live.

Things that bite people

A few patterns I've seen catch teams out repeatedly.

Underestimating the operational cost of streaming:

Streaming infrastructure has more moving parts than batch. There's more to monitor. There's more to debug at odd hours. There's more state to manage. Teams that jump into streaming without planning for the operational burden often end up regretting it.

Forgetting about ordering and exactly-once semantics:

Events can arrive out of order. They can be duplicated. They can be lost. Different stream processors handle these issues with different guarantees. Understand what your system promises before you depend on it for anything critical.

Ignoring backpressure:

When downstream systems can't keep up with the rate of events flowing in, something has to give. Either you slow down the producers, buffer the events, or drop them. Designing for backpressure from day one is much easier than retrofitting it.

Not budgeting for state size:

Stateful stream processing accumulates data. Sessions, aggregations, joins, all of these eat memory and disk. Watch the state size in your monitoring and plan capacity accordingly. Many Flink jobs crashed because nobody noticed the state had grown to fifty gigabytes.

Trying to do everything with one tool:

There's a temptation to pick a single streaming platform and make it solve every data problem you have. Resist this. Use the right tool for each job. Kafka for transport. Flink or Spark for heavy processing. Debezium for CDC. Airflow for orchestration. The boundaries between these tools exist for good reasons.

What you can do to begin

If you're new to streaming and want to try it, here's a path I'd suggest.

This weekend, spin up Kafka with Docker and play with it. Write a producer that generates fake events. Write a consumer that reads them. Feel how the basic primitives work. An hour of this teaches you more than any tutorial.

Next week, pick one real use case in your work and ask whether it could benefit from streaming. Not whether it could be done with streaming. Whether the business value of fresher data is worth the engineering cost.

If the answer is yes, build the smallest possible proof of concept. Don't try to migrate your whole pipeline. Take one feed of events, process them with Flink or Spark Structured Streaming, and write the results somewhere. See what the latency looks like. See what the operational experience is like.

Where things are heading

A few trends worth watching in the data integration space.

The line between streaming and batch keeps fading. Apache Beam was an early attempt at unifying the two APIs. dbt is moving in this direction. Spark Structured Streaming and Flink